

XML-документация (Visual C++)

В Visual C++ можно добавить комментарии в исходный код, обрабатываемый в XML-файл документации. Затем этот файл можно использовать в качестве входного для процесса, создающего документацию для классов в коде.

В файле кода Visual C++ комментарии к XML-документации должны находиться непосредственно перед определением метода или типа. Комментарии можно использовать для заполнения подсказок по данным для кратких сведений IntelliSense в следующих сценариях:

- 1. Когда код компилируется как компонент среда выполнения Windows с файлом WINMD
- 2. Если исходный код включен в текущий проект
- 3. В библиотеке, объявления типов и реализации которой находятся в том же файле заголовка

 Примечание.

В текущем выпуске комментарии кода не обрабатываются в шаблонах или ничего, содержащего тип шаблона (например, функция, принимающая параметр в качестве шаблона). Добавление таких комментариев приведет к неопределенному поведению.

Дополнительные сведения о создании XML-файла с комментариями документации см. в следующих статьях.

 Развернуть таблицу

Сведения	Смотрите
Используемые параметры компилятора	/doc
Теги, позволяющие реализовать часто используемые функции в документации	Рекомендуемые теги для комментариев документации
Строки идентификаторов, создаваемые компилятором для идентификации конструкций в коде	Обработка XML-файла
Разделение тегов документации	Разделители для тегов документации по Visual C++
Создание XML-файла из одного или нескольких XDC-	Справочник по XDCMake

Сведения	Смотрите
файлов.	
Ссылки на сведения об XML, посвященные связи с областями компонентов Visual Studio	XML в Visual Studio

Если вам нужно поместить специальные символы XML в текст комментария документации, следует использовать сущности XML или раздел CDATA.

См. также

[Расширения компонентов для платформ среды выполнения](#)

Last updated on 07.11.2025

Рекомендуемые теги для комментариев документации

Компилятор MSVC обрабатывает комментарии документации в коде, чтобы создать XDC-файл для каждого скомпилированного исходного файла. Затем `xdcmake.exe` обрабатывает XDC-файлы для создания XML-файла документации. Обработка XML-файла для создания документации — это подробные сведения, которые необходимо реализовать на сайте.

Теги обрабатываются в конструкциях, таких как типы и их члены.

Теги должны стоять прямо перед типами или членами.

⚠ Примечание.

Комментарии к документации нельзя применять к пространству имен или вне определений строк для свойств и событий; Комментарии к документации должны находиться в объявлениях в классе.

Компилятор обрабатывает любые теги, имеющие допустимый формат XML. С помощью следующих тегов реализуются часто используемые в пользовательской документации возможности:

`<c>`
`<code>`
`<example>`
`<exception>`¹
`<include>`¹
`<list>`
`<para>`
`<param>`¹
`<paramref>`¹
`<permission>`¹
`<remarks>`
`<returns>`
`<see>`¹
`<seealso>`¹
`<summary>`
`<value>`

1. Компилятор проверяет синтаксис.

В текущем выпуске компилятор MSVC не поддерживает `<paramref>` тег, поддерживаемый другими компиляторами Visual Studio. Возможно, поддержка `<paramref>` будет добавлена в будущих выпусках Visual C++.

См. также

[XML-документация](#)

Last updated on 07.11.2025

<c> тег документации

Тег `<c>` указывает, что текст в описании должен быть помечен как код. Чтобы определить несколько строк в качестве кода, используйте тег `<code>`.

Синтаксис

C++

```
/// <c>text</c>
```

Параметры

`text`

Текст, который нужно указать в качестве кода.

Замечания

Скомпилируйте их для [/doc](#) обработки примечаний документации к файлу.

Пример

C++

```
// xml_c_tag.cpp
// compile with: /doc /LD
// post-build command: xdcmake xml_c_tag.xdc

/// Text for class MyClass.
class MyClass {
public:
    int m_i;
    MyClass() : m_i(0) {}

    /// <summary><c>MyMethod</c> is a method in the <c>MyClass</c> class.
    /// </summary>
    int MyMethod(MyClass * a) {
        return a -> m_i;
    }
};
```

См. также

Last updated on 07.11.2025

`<code>` тег документации

Тег `<code>` позволяет указать одну или несколько строк в виде кода.

Синтаксис

C++

```
/// <code>content</code>
```

```
/// <code>  
/// content  
/// content  
/// </code>
```

Параметры

`content`

Текст, который необходимо пометить как код.

Замечания

Используется `<c>` для указания части текста, который должен быть помечен как код.

Скомпилируйте их для [/doc](#) обработки примечаний документации к файлу.

Пример

Пример использования тега `<code>` см. в разделе [<example>](#).

См. также

[XML-документация](#)

<example> тег документации

Тег `<example>` позволяет указать пример использования метода или другого элемента библиотеки. Как правило, использование этого тега также будет включать `<code>` тег.

Синтаксис

C++

```
/// <example>description</example>
```

Параметры

description

Описание примера кода.

Замечания

Скомпилируйте их для `/doc` обработки примечаний документации к файлу.

Пример

C++

```
// xml_example_tag.cpp
// compile with: /clr /doc /LD
// post-build command: xdcmake xml_example_tag.dll

/// Text for class MyClass.
public ref class MyClass {
public:
    /// <summary>
    /// GetZero method
    /// </summary>
    /// <example> This sample shows how to call the GetZero method.
    /// <code>
    /// int main()
    /// {
    ///     return GetZero();
    /// }
    /// </code>
    /// </example>
    static int GetZero() {
        return 0;
    }
};
```



```
}  
};
```

См. также

[XML-документация](#)

Last updated on 07.11.2025

<exception> тег документации

Тег `<exception>` служит для указания возможных исключений. Этот тег применяется к определению метода.

Синтаксис

C++

```
///<exception cref="member">description</exception>
```

Параметры

`member`

Ссылка на исключение, доступное из текущей среды компиляции. Используя правила подстановки имен, компилятор проверяет, существует ли исключение, и приводит `member` к каноническому имени элемента в выходных XML-данных. Компилятор выдает предупреждение, если он не находит `member`.

Заключите имя в одинарные или двойные кавычки.

Дополнительные сведения о создании ссылки на универсальный `cref` тип см. в разделе [<see>](#).

`description`

Описание.

Замечания

Скомпилируйте их для [/doc](#) обработки примечаний документации к файлу.

Компилятор MSVC пытается разрешить `cref` ссылки в одном проходе по комментариям документации. Если используется правила подстановки C++, если символ не найден компилятором, ссылка помечается как неразрешенная. Дополнительные сведения см. в разделе [<seealso>](#).

Пример

C++

```
// xml_exception_tag.cpp
// compile with: /clr /doc /LD
// post-build command: xdcmake xml_exception_tag.dll
using namespace System;

/// Text for class EClass.
public ref class EClass : public Exception {
    // class definition ...
};

/// <exception cref="System.Exception">Thrown when... .</exception>
public ref class TestClass {
    void Test() {
        try {
        }
        catch(EClass^) {
        }
    }
};
```

См. также

[XML-документация](#)

Last updated on 07.11.2025

<include> тег документации

Тег `<include>` позволяет задать ссылку на комментарии в другом файле, которые описывают типы и элементы вашего исходного кода. Этот тег является альтернативой размещению комментариев документации непосредственно в файле исходного кода. Например, можно использовать `<include>` для вставки стандартных комментариев", которые используются во всей команде или компании.

Синтаксис

C++

```
/// <include file='filename' path='tag-path[@name="ID"' />
```

Параметры

filename

Имя файла, содержащего документацию. Имя файла может быть дополнено с указанием пути. Заключите имя в одинарные или двойные кавычки. Компилятор выдает предупреждение, если он не находит *filename*.

tag-path

Допустимое выражение XPath, которое выбирает нужный набор узлов, содержащийся в файле.

name

Спецификатор имени в теге, предшествующий комментариям. *name* будет иметь идентификатор *ID*.

ID

Идентификатор тега, который предшествует комментариям. Заключите идентификатор в одинарные или двойные кавычки.

Замечания

Тег `<include>` использует XML-синтаксис XPath. Сведения о способах настройки с помощью `<include>` см. в документации по XPath.

Скомпилируйте их для [/doc](#) обработки примечаний документации к файлу.

Пример

В этом примере используется несколько файлов. Первый файл, который использует, `<include>` содержит следующие комментарии к документации:

C++

```
// xml_include_tag.cpp
// compile with: /clr /doc /LD
// post-build command: xdcmake xml_include_tag.dll

/// <include file='xml_include_tag.doc' path='MyDocs/MyMembers[@name="test"]/*' />
public ref class Test {
    void TestMethod() {
    }
};

/// <include file='xml_include_tag.doc' path='MyDocs/MyMembers[@name="test2"]/*' />
public ref class Test2 {
    void Test() {
    }
};
```

Второй файл содержит `xml_include_tag.doc` следующие комментарии к документации:

XML

```
<MyDocs>

<MyMembers name="test">
<summary>
The summary for this type.
</summary>
</MyMembers>

<MyMembers name="test2">
<summary>
The summary for this other type.
</summary>
</MyMembers>

</MyDocs>
```

Выходные данные программы

XML

```
<?xml version="1.0"?>
<doc>
```

```
<assembly>
  <name>t2</name>
</assembly>
<members>
  <member name="T:Test">
    <summary>
The summary for this type.
</summary>
    </member>
    <member name="T:Test2">
      <summary>
The summary for this other type.
</summary>
    </member>
  </members>
</doc>
```

См. также

[XML-документация](#)

Last updated on 07.11.2025

<list> и <listheader> теги документации

Блок `<listheader>` используется для определения строки заголовка в таблице или списке определений. При определении таблицы необходимо ввести данные для термина в заголовке.

Синтаксис

XML

```
<list type="bullet" | "number" | "table">
  <listheader>
    <term>term</term>
    <description>description</description>
  </listheader>
  <item>
    <term>term</term>
    <description>description</description>
  </item>
</list>
```

Параметры

`term`

Термин, который будет определен в `description`.

`description`

Либо элемент маркированного или нумерованного списка, либо определение `term`.

Замечания

Каждый элемент в списке указывается в блоке `<item>`. При создании списка определений необходимо указать одновременно `term` и `description`. Тем не менее для таблицы, маркированного или нумерованного списка достаточно ввести только `description`.

Число блоков `<item>` в списке или таблице не ограничено.

Скомпилируйте их для [/doc](#) обработки примечаний документации к файлу.

Пример

C++

```
// xml_list_tag.cpp
// compile with: /doc /LD
// post-build command: xdcmake xml_list_tag.dll
/// <remarks>Here is an example of a bulleted list:
/// <list type="bullet">
/// <item>
/// <description>Item 1.</description>
/// </item>
/// <item>
/// <description>Item 2.</description>
/// </item>
/// </list>
/// </remarks>
class MyClass {};
```

См. также

[XML-документация](#)

Last updated on 07.11.2025

<para> тег документации

Тег `<para>` используется внутри другого тега, например [<summary>](#), [<remarks>](#) или [<returns>](#), и позволяет добавить к тексту структуру.

Синтаксис

C++

```
/// <para>content</para>
```

Параметры

`content`

Текст абзаца.

Замечания

Скомпилируйте их для [/doc](#) обработки примечаний документации к файлу.

Пример

См. [<summary>](#) с примером использования `<para>`.

См. также

[XML-документация](#)

Last updated on 07.11.2025

<param> тег документации

Тег `<param>` следует использовать в комментариях к объявлению метода для описания одного из параметров такого метода.

Синтаксис

C++

```
///<param name='param-name'>description</param>
```

Параметры

`param-name`

Имя параметра метода. Заключите имя в одинарные или двойные кавычки. Компилятор выдает предупреждение, если он не находит `param-name`.

`description`

Описание параметра.

Замечания

Текст тега `<param>` будет отображаться в IntelliSense, [браузере](#) объектов и в веб-отчете о комментариях кода.

Скомпилируйте их для [/doc](#) обработки примечаний документации к файлу.

Пример

C++

```
// xml_param_tag.cpp
// compile with: /clr /doc /LD
// post-build command: xdcmake xml_param_tag.dll
/// Text for class MyClass.
public ref class MyClass {
    ///<param name="Int1">Used to indicate status.</param>
    void MyMethod(int Int1) {
    }
};
```

См. также

[XML-документация](#)

Last updated on 07.11.2025

<paramref> тег документации

Тег `<paramref>` позволяет указать, что слово является параметром. XML-файл можно обрабатывать таким образом, чтобы отформатировать этот параметр по-разному.

Синтаксис

C++

```
/// <paramref name="ref-name"/>
```

Параметры

ref-name

Имя параметра, на который указывается ссылка. Заключите имя в одинарные или двойные кавычки. Компилятор выдает предупреждение, если он не находит *ref-name*.

Замечания

Скомпилируйте их для [/doc](#) обработки примечаний документации к файлу.

Пример

C++

```
// xml_paramref_tag.cpp
// compile with: /clr /doc /LD
// post-build command: xdcmake xml_paramref_tag.dll
/// Text for class MyClass.
public ref class MyClass {
    /// <summary>MyMethod is a method in the MyClass class.
    /// The <paramref name="Int1"/> parameter takes a number.
    /// </summary>
    void MyMethod(int Int1) {}
};
```

См. также

[XML-документация](#)

<permission> тег документации

Тег `<permission>` позволяет документировать уровень доступа для элемента. [PermissionSet](#) позволяет задать уровень доступа для члена.

Синтаксис

C++

```
/// <permission cref="member">description</permission>
```

Параметры

`member`

Ссылка на член или поле, которые доступны для вызова из текущей среды компиляции. Компилятор проверяет, существует ли элемент кода, и приводит `member` к каноническому имени элемента в выходных XML-данных. Заключите имя в одинарные или двойные кавычки.

Компилятор выдает предупреждение, если он не находит `member`.

Сведения о создании ссылки на универсальный `cref` тип см. в разделе [<see>](#).

`description`

Описание уровня доступа для члена.

Замечания

Скомпилируйте их для [/doc](#) обработки примечаний документации к файлу.

Компилятор MSVC попытается разрешить `cref` ссылки в одном проходе через комментарии документации. Если компилятор не находит символ при использовании правил подстановки C++, ссылка будет помечена как неразрешенная. Дополнительные сведения см. в разделе [<seealso>](#).

Пример

C++

```
// xml_permission_tag.cpp  
// compile with: /clr /doc /LD
```

```
// post-build command: xdcmake xml_permission_tag.dll
using namespace System;
/// Text for class TestClass.
public ref class TestClass {
    /// <permission cref="System::Security::PermissionSet">Everyone can access this
    method.</permission>
    void Test() {}
};
```

См. также

[XML-документация](#)

Last updated on 07.11.2025

<remarks> тег документации

Тег `<remarks>` позволяет добавить сведения о типе, дополняющие информацию, указанную с помощью тега `<summary>`. Эта информация отображается в [обозревателе объектов](#) и в веб-отчете комментариев кода.

Синтаксис

C++

```
/// <remarks>description</remarks>
```

Параметры

description

Описание элемента.

Замечания

Скомпилируйте их для [/doc](#) обработки примечаний документации к файлу.

Пример

C++

```
// xml_remarks_tag.cpp
// compile with: /LD /clr /doc
// post-build command: xdcmake xml_remarks_tag.dll

using namespace System;

/// <summary>
/// You may have some primary information about this class.
/// </summary>
/// <remarks>
/// You may have some additional information about this class.
/// </remarks>
public ref class MyClass {};
```

См. также

[XML-документация](#)

Last updated on 07.11.2025

<returns> тег документации

Тег `<returns>` следует использовать в комментариях к объявлению метода для описания возвращаемого значения.

Синтаксис

C++

```
/// <returns>description</returns>
```

Параметры

description

Описание возвращаемого значения.

Замечания

Скомпилируйте их для [/doc](#) обработки примечаний документации к файлу.

Пример

C++

```
// xml_returns_tag.cpp
// compile with: /LD /clr /doc
// post-build command: xdcmake xml_returns_tag.dll

/// Text for class MyClass.
public ref class MyClass {
public:
    /// <returns>Returns zero.</returns>
    int GetZero() { return 0; }
};
```

См. также

[XML-документация](#)

<see> тег документации

Тег `<see>` позволяет задать ссылку из текста. Используйте `<seealso>` для указания текста, который может появиться в разделе "См. также".

Синтаксис

C++

```
/// <see cref="member"/>
```

Параметры

`member`

Ссылка на член или поле, которые доступны для вызова из текущей среды компиляции. Заключите имя в одинарные или двойные кавычки.

Компилятор проверяет, существует ли элемент кода, и разрешает `member` в имя элемента в выходных XML-данных. Компилятор выдает предупреждение, если он не находит `member`.

Замечания

Скомпилируйте их для `/doc` обработки примечаний документации к файлу.

Пример использования `<see>` см. в разделе `<summary>`.

Компилятор MSVC попытается разрешить `cref` ссылки в одном проходе через комментарии документации. Если компилятор не находит символ при использовании правил подстановки C++, он помечает ссылку как неразрешенную. Дополнительные сведения см. в разделе `<seealso>`.

Пример

В следующем примере показано, как можно ссылаться `cref` на универсальный тип, чтобы компилятор разрешал ссылку.

C++

```
// xml_see_cref_example.cpp
// compile with: /LD /clr /doc
// the following cref shows how to specify the reference, such that,
```

```
// the compiler will resolve the reference
/// <see cref="C{T}">
/// </see>
ref class A {};

// the following cref shows another way to specify the reference,
// such that, the compiler will resolve the reference
/// <see cref="C < T >" />

// the following cref shows how to hard-code the reference
/// <see cref="T:C`1">
/// </see>
ref class B {};

/// <see cref="A">
/// </see>
/// <typeparam name="T"></typeparam>
generic<class T>
ref class C {};
```

См. также

[XML-документация](#)

Last updated on 07.11.2025

<seealso> тег документации

С помощью тега `<seealso>` можно указать текст, который должен отображаться в разделе "См. также". Тег `<see>` позволяет задать ссылку из текста.

Синтаксис

C++

```
///<seealso cref="member"/>
```

Параметры

`member`

Ссылка на член или поле, которые доступны для вызова из текущей среды компиляции. Заключите имя в одинарные или двойные кавычки.

Компилятор проверяет, существует ли элемент кода, и разрешает `member` в имя элемента в выходных XML-данных. Компилятор выдает предупреждение, если он не находит `member`.

Сведения о создании ссылки на универсальный `cref` тип см. в разделе `<see>`.

Замечания

Скомпилируйте их для `/doc` обработки примечаний документации к файлу.

См. `<summary>` с примером использования `<seealso>`.

Компилятор MSVC пытается разрешить `cref` ссылки в одном проходе по комментариям документации. Если компилятор не находит символ при использовании правил подстановки C++, он помечает ссылку как неразрешенную.

Пример

В следующем примере неразрешенный символ ссылается на `cref`. XML-комментарий для объекта `cref A::Test` хорошо сформирован: `<seealso cref="M:A.Test" />` Тем не менее, `cref` становится `<seealso cref="!:B::Test" /> B::Test`.

C++

```
// xml_seealso_tag.cpp
// compile with: /LD /clr /doc
// post-build command: xdcmake xml_seealso_tag.dll

/// Text for class A.
public ref struct A {
    /// <summary><seealso cref="A::Test"/>
    /// <seealso cref="B::Test"/>
    /// </summary>
    void MyMethod(int Int1) {}

    /// text
    void Test() {}
};

/// Text for class B.
public ref struct B {
    void Test() {}
};
```

См. также

[XML-документация](#)

Last updated on 07.11.2025

<summary>

Тег `<summary>` следует использовать для описания типа или элемента типа. Чтобы добавить дополнительную информацию в описание типа, используйте `<remarks>`.

Синтаксис

C++

```
/// <summary>description</summary>
```

Параметры

description

Сводка объекта.

Замечания

Текст `<summary>` тега является единственным источником сведений о типе в IntelliSense, а также отображается в [браузере](#) объектов и в веб-отчете о комментариях кода.

Скомпилируйте их для [/doc](#) обработки примечаний документации к файлу.

Пример

C++

```
// xml_summary_tag.cpp
// compile with: /LD /clr /doc
// post-build command: xdcmake xml_summary_tag.dll

/// Text for class MyClass.
public ref class MyClass {
public:
    /// <summary>MyMethod is a method in the MyClass class.
    /// <para>Here's how you could make a second paragraph in a description. <see
    cref="System::Console::WriteLine"/> for information about output statements.</para>
    /// <seealso cref="MyClass::MyMethod2"/>
    /// </summary>
    void MyMethod(int Int1) {}

    /// text
    void MyMethod2() {}
};
```

См. также

[XML-документация](#)

Last updated on 07.11.2025

<value> тег документации

Тег `<value>` позволяет описать методы доступа к свойству и свойствам. При добавлении свойства с помощью мастера кода в интегрированной среде разработки Visual Studio он добавит `<summary>` тег для нового свойства. Необходимо вручную добавить `<value>` тег, чтобы описать значение, которое представляет свойство.

Синтаксис

C++

```
/// <value>property-description</value>
```

Параметры

`property-description`

Описание свойства.

Замечания

Скомпилируйте их для `/doc` обработки примечаний документации к файлу.

Пример

C++

```
// xml_value_tag.cpp
// compile with: /LD /clr /doc
// post-build command: xdcmake xml_value_tag.dll
using namespace System;
/// Text for class Employee.
public ref class Employee {
private:
    String ^ name;
    /// <value>Name accesses the value of the name data member</value>
public:
    property String ^ Name {
        String ^ get() {
            return name;
        }
        void set(String ^ i) {
            name = i;
        }
    }
}
```

```
}  
};
```

См. также

[XML-документация](#)

Last updated on 07.11.2025

Обработка XML-файлов документации

Компилятор создает строку идентификатора для каждой конструкции в коде, помеченной для создания документации. Дополнительные сведения см. [в комментариях к рекомендуемой документации](#) по тегам. Строка идентификатора однозначно определяет конструкцию. Программы, обрабатывающие XML-файл, могут использовать строку идентификатора для идентификации соответствующего платформа .NET Framework метаданных или элемента отражения, к которому применяется документация.

XML-файл не является иерархическим представлением кода, это плоский список с созданным идентификатором для каждого элемента.

Компилятор соблюдает следующие правила при формировании строк идентификаторов.

- Пробелы в строку не вставляются.
- Первая часть строки идентификатора определяет тип идентифицируемого члена. Это один символ, за которым следует двоеточие. Используются следующие типы элементов.

 Развернуть таблицу

Символ	Description
N	Пространство имен Невозможно добавить комментарии к документации в пространство имен, ссылки на пространство имен возможны.
T	Тип: класс, интерфейс, структура, перечисление, делегат
D	Typedef
F	Поле
P	Свойство (включая индексаторы или другие индексированные свойства)
Пн.	Метод (включая такие специальные методы, как конструкторы, операторы и т. д.)
E	Мероприятие
!	Строка ошибки Остальная часть строки предоставляет сведения об ошибке. Компилятор MSVC создает сведения об ошибке для ссылок, которые не могут быть разрешены.

- Вторая часть строки содержит полное имя элемента, начиная от корня пространства имен. Имя элемента, включающие типы и пространство имен разделяются точками.

Если в имени самого элемента есть точки, они заменяются символами решетки ("#"). Предполагается, что в именах элементов не может содержаться символ решетки. Например, полное имя конструктора `String` будет `System.String.#ctor`.

- Для свойств и методов, имеющих аргументы, список этих аргументов заключается в круглые скобки и указывается в конце. Если аргументы не используются, скобки отсутствуют. Аргументы разделяются запятыми. Каждый аргумент закодирован так же, как кодируется в сигнатуре платформа .NET Framework:
 - Базовые типы. Обычные типы (`ELEMENT_TYPE_CLASS` или `ELEMENT_TYPE_VALUETYPE`) представляются в виде полного имени типа.
 - Встроенные типы (например, `ELEMENT_TYPE_I4`, , `ELEMENT_TYPE_STRING` `ELEMENT_TYPE_OBJECT` `ELEMENT_TYPE_TYPEDBYREF`, и `ELEMENT_TYPE_VOID`) представляются как полное имя соответствующего полного типа, например `System.Int32` или `.System.TypedReference`
 - `ELEMENT_TYPE_PTR` представляется в виде "*" после имени измененного типа.
 - `ELEMENT_TYPE_BYREF` представляется в виде "@" после имени измененного типа.
 - `ELEMENT_TYPE_PINNED` представляется в виде "^" после имени измененного типа. Компилятор MSVC никогда не создает этот элемент.
 - `ELEMENT_TYPE_CMOD_REQ` представляется как "!" и полное имя класса модификатора после измененного типа. Компилятор MSVC никогда не создает этот элемент.
 - `ELEMENT_TYPE_CMOD_OPT` представляется как "!" и полное имя класса модификатора после измененного типа.
 - `ELEMENT_TYPE_SZARRAY` представляется как "[]" после типа элемента массива.
 - `ELEMENT_TYPE_GENERICARRAY` представляется как "[?]" после типа элемента массива. Компилятор MSVC никогда не создает этот элемент.
 - `ELEMENT_TYPE_ARRAY` представляется как "[нижний размер нижней границы] : , : ,", где число запятых — 1, а нижняя граница и размер каждого измерения, если известно, представлены в десятичном разряде. Если нижняя граница или размер не указаны, они опускаются. Если нижнюю границу и размер опускают для определенного измерения, то ":" также опускается. Например, 2-мерный массив с 1 нижней границей и неопределенными размерами представлен как `[1:,1:]`.
 - `ELEMENT_TYPE_FNPTR` представляется как "=FUNC: `type` (`подпись`)", где `type` обозначает возвращаемый тип, а `подпись` представляет аргументы метода. Если

аргументы не используются, скобки опускаются. Компилятор MSVC никогда не создает этот элемент.

Следующие компоненты подписи не представлены, так как они никогда не используются для разных перегруженных методов:

- Соглашение о вызовах
- Возвращаемый тип
- `ELEMENT_TYPE_SENTINEL`
- Только для операторов преобразования возвращаемое значение метода закодировано как "`~`", за которым следует тип возвращаемого значения, как было закодировано ранее.
- Для универсальных типов имя типа завершается символом обратного апострофа и числом, которое обозначает количество параметров универсального типа. Например,

XML

```
<member name="T:MyClass`2">
```

В примере показан тип, определенный как `public class MyClass<T, U>`.

Для методов, которые принимают универсальные типы в качестве параметров, параметры универсального типа указываются в виде чисел, предопределенных обратными галками (например, 0, 1). Каждое число представляет позицию массива на основе нуля для универсальных параметров типа.

Пример

Примеры ниже демонстрируют, как создаются строки идентификатора для класса и его элементов.

C++

```
// xml_id_strings.cpp
// compile with: /clr /doc /LD
///
namespace N {
// "N:N"

    /// <see cref="System" />
    // <see cref="N:System"/>
```

```

ref class X {
// "T:N.X"

protected:
    ///
    !X(){}
    // "M:N.X.Finalize", destructor's representation in metadata

public:
    ///
    X() {}
    // "M:N.X.#ctor"

    ///
    static X() {}
    // "M:N.X.#cctor"

    ///
    X(int i) {}
    // "M:N.X.#ctor(System.Int32)"

    ///
    ~X() {}
    // "M:N.X.Dispose", Dispose function representation in metadata

    ///
    System::String^ q;
    // "F:N.X.q"

    ///
    double PI;
    // "F:N.X.PI"

    ///
    int f() { return 1; }
    // "M:N.X.f"

    ///
    int bb(System::String ^ s, int % y, void * z) { return 1; }
    // "M:N.X.bb(System.String,System.Int32@,System.Void*)"

    ///
    int gg(array<short> ^ array1, array< int, 2 >^ IntArray) { return 0; }
    // "M:N.X.gg(System.Int16[], System.Int32[0:,0:])"

    ///
    static X^ operator+(X^ x, X^ xx) { return x; }
    // "M:N.X.op_Addition(N.X,N.X)"

    ///
    property int prop;
    // "M:N.X.prop"

    ///
    property int prop2 {

```

```

// "P:N.X.prop2"

///
int get() { return 0; }
// M:N.X.get_prop2

///
void set(int i) {}
// M:N.X.set_prop2(System.Int32)
}

///
delegate void D(int i);
// "T:N.X.D"

///
event D ^ d;
// "E:N.X.d"

///
ref class Nested {};
// "T:N.X.Nested"

///
static explicit operator System::Int32 (X x) { return 1; }
//
"M:N.X.op_Explicit(N.X!System.Runtime.CompilerServices.IsByValue)~System.Int32"
};
}

```

См. также

[XML-документация](#)

Last updated on 07.11.2025

Разделители для тегов документации по Visual C++

Для использования тегов документации требуются *разделители*, указывающие компилятору, где начинается и заканчивается комментарий документации.

С тегами в XML-документации можно использовать следующие виды разделителей:

[🔗 Развернуть таблицу](#)

Разделитель	Description
<code>///</code>	Это форма, показанная в примерах документации и используемая шаблонами проектов Visual Studio C++.
<code>/** */</code>	Это многострочные разделители.

На использование разделителей `/** */` распространяется несколько правил форматирования.

- Для строки, содержащей `/**` разделитель, если остальная часть строки является пробелами, строка не обрабатывается для комментариев. Если первый символ является пробелом, этот символ пробела игнорируется, а остальная часть строки обрабатывается. В противном случае весь текст, расположенный в строке после разделителя `/**`, обрабатывается как часть комментария.
- Если в строке, содержащей `*/` разделитель, есть только пробелы до `*/` разделителя, эта строка игнорируется. В противном случае текст строки, расположенный перед разделителем `*/`, обрабатывается как часть комментария, и к нему применяются правила сопоставления шаблонов, описанные в следующем разделе.
- Для строк после того, как он начинается с `/**` разделителя, компилятор ищет общий шаблон в начале каждой строки, состоящей из необязательных пробелов и звездочки (*), за которым следует больше дополнительного пробела. Если компилятор находит общий набор символов в начале каждой строки, он игнорирует этот шаблон для всех строк после разделителя `/**` вплоть до самой строки, содержащей разделитель `*/`, и, возможно, включая ее.

Примеры

- В следующем комментарии будет обработана только строка, которая начинается с `<summary>`. Следующие форматы с двумя тегами позволяют создать точно такие же

комментарии.

C++

```
/**  
<summary>text</summary>  
*/  
/** <summary>text</summary> */
```

- Компилятор применяет шаблон " * " для игнорирования в начале второй и третьей строки.

C++

```
/**  
 * <summary>  
 * text </summary>*/
```

- Компилятор не находит шаблон в этом комментарии, так как на второй строке нет звездочки. Весь текст во второй и третьей строках до тех пор, пока не */ будет обработан в рамках комментария.

C++

```
/**  
 * <summary>  
 text </summary>*/
```

- Компилятор не обнаруживает шаблон в этом комментарии по двум причинам. Во-первых, нет строки, начинающейся с согласованного количества пробелов перед звездочкой. Во-вторых, пятая строка начинается с символа табуляции, который не является пробелом. Весь текст из второй строки, пока не */ будет обработан как часть комментария.

C++

```
/**  
 * <summary>  
 * text  
 * text2  
 * </summary>  
*/
```

См. также

Last updated on 07.11.2025